



Indian Institute of Technology Kanpur
Department of Computer Science and Engineering

CovertTux: A Cache Policy-oblivious Timing Covert Channel on a Linux System

Dhruv Gupta - BT/CSE/220361

Pragati Agrawal - BT/CSE/220779

Supervisors: Prof. Mainak Chaudhuri & Prof. Debadatta Mishra

Mentor: Ms. Yashika Verma

April 25, 2025

Acknowledgements

We extend our heartfelt gratitude to *Prof. Mainak* and *Prof. Debadatta* for their invaluable guidance and unwavering support throughout this project, especially for their commitment to regular meetings despite their busy schedules.

We also wish to express our sincere thanks to our mentor, *Ms. Yashika*, for her in-depth knowledge and continuous assistance, which greatly enriched our understanding of the subject matter. This work originated from her research work ***LeakyRand***. Additionally, we appreciate the cooperation of the CSE department staff, whose logistical support was essential to the success of our project.

Abstract

The last-level cache (LLC) in a chip-multiprocessor is typically shared between the cores of the processor. Such a shared LLC has been shown to be vulnerable against timing channel-based security attacks that exploit the latency difference between an LLC hit and an LLC miss. Two processes (sender and receiver) co-scheduled on two cores of a chip-multiprocessor can exploit this latency difference to establish a covert channel, an illegitimate and covert way of leaking sensitive information. Last semester we presented CovertCraft, a proof-of-concept implementation of a timing-based covert channel exploiting a fully associative cache with random replacement policy synthesized on a field programmable gate array (FPGA) in an ideal noiseless environment. Since a set of a set associative cache can be viewed as a standalone fully associative cache, it is possible to mount the same attack on a set associative shared LLC of a commodity chip-multiprocessor. To demonstrate the feasibility of this attack, we present CovertTux, an implementation of our timing-based covert channel attack adopted to a set associative LLC of a commodity server running off-the-shelf Linux operating system. An interesting property of this attack that distinguishes it from most timing-based attacks on set associative caches is that it does not need to know or reverse-engineer the replacement policy exercised by the LLC. This advantage arises from the fact that our covert channel, by design, works with a random replacement policy. However, establishing such a channel on a real system poses several challenges such as dealing with system noise, finding independent eviction sets for the receiver and the sender which map to the same, yet relatively less noisy, LLC set, and properly synchronizing the sender and receiver processes without using any shared memory. Our covert channel implemented on a set of the 16 MB 16-way set associative LLC of an Intel E-2278G Xeon processor clocked nominally at 3.4 GHz running ubuntu 18.04 achieves a communication bandwidth varying between 30 Kbps and 570 Kbps depending on different parameter settings, while maintaining a bit error rate ranging from 8.7×10^{-4} to 4.5×10^{-2} , averaging around 3.3×10^{-3} .

Contents

1	Introduction	1
1.1	Our Machine	1
1.2	Overview	2
2	Finding Threshold	3
2.1	Bypassing the Prefetcher	3
2.2	Bypassing L1 and L2 Hit Times	4
2.3	Collecting Miss Timings	4
2.4	Collecting Hit Timings	5
2.5	Threshold for our machine	6
3	Receiver: Finding Eviction Set	7
3.1	Data Structures & Variables	7
3.2	Basic Idea	8
3.3	FSM of Eviction Set Finding	8
3.4	Algorithm to find Eviction Set	9
3.5	Algorithm to Construct Occupancy Set	11
3.6	Algorithm to Test if a set is an eviction set	12
3.7	Validating Eviction Set	13
4	Sender: Finding Disturbance Set	14
4.1	Algorithm to find Disturbance Set	14
4.2	FSM for finding the Disturbance set	15
5	Validating Disturbance Set & Eviction Set	16
6	Unrolling Factor Threshold	17
6.1	What is Unrolling Factor?	17
6.2	Finding Threshold for any Unrolling Factor	17
6.3	FSM for finding this threshold	18
7	Communication	19
7.1	Basic Idea	19
7.2	FSM for communication	20
8	Assumptions & Scope of Improvement	21
8.1	Synchronisation	21
8.2	Privileges	21

9	Evaluation Metrics	22
9.1	Data Collection	22
9.2	Bandwidth	22
9.2.1	Calculation	22
9.2.2	Results & Inferences	23
9.3	BER	24
9.3.1	Calculation	24
9.3.2	Results & Inferences	24
9.4	Is large UF better?	25
	Glossary	26

1. Introduction

A covert channel is an illegitimate medium of communication between two entities, where information is transferred through a method not originally intended or designed for communication. Researches have shown that such covert channels can be created in a multi-core system by exploiting the resources which are shared across the cores. In multi-core systems, the last level cache (LLC) is typically shared across the cores. Two processes can exploit the latency difference between a LLC hit and a LLC miss to set up a covert channel and use it to leak sensitive information. Researches have discovered multiple strategies to establish such covert channels. Prime+Probe is one such strategy[1].

In our last UGP we presented CovertCraft which aimed to provide a proof-of-concept implementation of a timing-based covert channel which exploits a fully associative cache with random replacement policy.

In this project, we aim to demonstrate the feasibility of this channel on a real system. We present our timing-based covert channel CovertTux, which is adopted to a set associative LLC of a commodity server running a Linux OS. Since, the channel is designed to work with random replacement policy, it would work with any replacement policy.

1.1 Our Machine

We have used one of IIT Kanpur's CSE server machines - 172.27.21.1 for our implementation. It is a Intel(R) Xeon(R) E-2278G CPU running at 3.40GHz having 16 logical cores. It has the following specifications -

- Cache Block Size : 64 B
- Page Size : 4 KB
- L1i Cache : 32 KB, 8-ways
- L1d Cache : 32 KB, 8-ways
- L2 Cache : 256 KB, 4-ways, non-inclusive
- L3 Cache (LLC) : 16 MB, 16-ways, **inclusive**
- L1 dTLB reach: 256 KB
- L2 TLB reach: 6MB

It is important to note that the channel needs an inclusive LLC to work. This is because if any block is evicted from the LLC by either the sender or the receiver process, it must also be evicted from the private caches (L1 and L2 caches) of the other process. Otherwise the effect of eviction would not be observed. This was an important criteria while choosing the machine.

1.2 Overview

We have created two versions of our channel:

- **Multi-threaded code:** In this case, the receiver process creates a thread which acts as the sender and the main thread works as the receiver. Since in this case address space is shared, the **disturbance-set** for sender is obtained as a subset of **eviction-set** elements found by the receiver. Also, shared flags (global variables) are used for synchronization. This design is not ideal since a typical covert channel attack works without the need of a shared memory. However it acts as an intermediate step to our final design.
- **Multi-process code:** In this case, the receiver process forks out a child process which acts as the sender. Since, in this case sender and receiver have different address space, there is an additional step to find the **disturbance-set** for the sender. Other things remain same as the multi-threaded version. For synchronization, we have utilized pipes, however, in an ideal scenario it can be (and should be) done by utilizing the worst case timings for each step.

In this report we primarily discuss the multi-process version of the channel. The multi-threaded version also works in the similar fashion, however the **disturbance-set** finding step is not needed in this case. Also, the synchronization is done differently as discussed above. In subsequent sections we discuss the steps involved in the implementation of our covert channel in order. At the end we discuss the results of our implementation and the performance of our covert channel. We also discuss the limitations of our implementation and scope of improvement. The diagram below shows the 5 major steps in creating our covert channel:

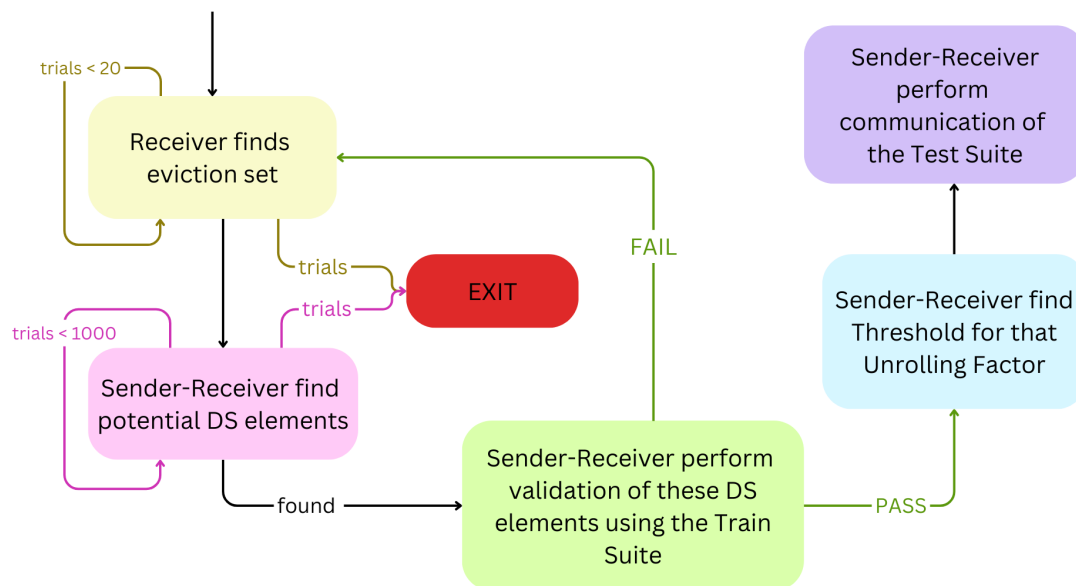


Figure 1.1: FSM showing the overview of CovertTux

2. Finding Threshold

For creating a timing based covert channel which exploits the latency difference between LLC hit and LLC miss, first step would be to find a suitable threshold value to differentiate between these two events. Let measured time be τ ,

$$\tau \leq \text{Threshold} \implies \text{LLC hit}$$

$$\tau > \text{Threshold} \implies \text{LLC miss}$$

Since these timings are non-deterministic in nature, there would be a band of LLC hit and miss latencies. If these bands are overlapping, there cannot be any value of threshold which can give 100% accuracy of classification. So we collected samples of LLC hit times and LLC miss times and then found a **THRESHOLD** as the median of the thresholds of all samples which would give minimum errors.

2.1 Bypassing the Prefetcher

The prefetcher is a hardware component that attempts to predict which data will be needed in the future and pre-loads it into the cache. This can interfere with our measurements, as it may cause cache hits to occur when we expect misses. To bypass the prefetcher, we can change our access pattern and access cache blocks across pages, because prefetchers generally do not identify access patterns across pages. Let us understand this by an example:

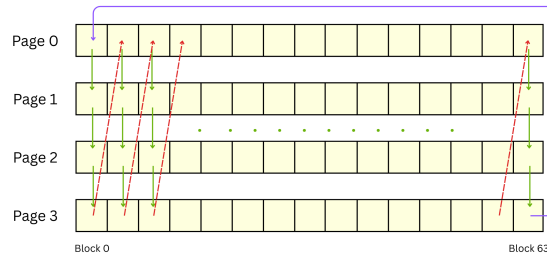


Figure 2.1: Bypassing the Prefetcher

In the example above, we have allocated 4 pages, each of size 4KB. Since a Cache Block size is 64B, each page will have 64 cache blocks. The prefetcher might try to prefetch the next cache block in the same page, but would not prefetch blocks from a different page to avoid the chance of TLB pollution. So we access blocks in the following order: first blocks of all the pages, then second block of all the pages and so on. If we reach the end, we start from the beginning in a cyclic manner.

The same access pattern is used throughout the project, to ensure we are indeed measuring what we intend to.

2.2 Bypassing L1 and L2 Hit Times

We want to develop a covert channel that uses the LLC miss and hit timings. Timing a miss is easy, as we can intentionally first flush the block using `clflush` instruction of x86 ISA and then reaccess it to measure the miss time.

For hit, we need to ensure that every timing we are measuring, must be only an LLC hit, and not an L1 or L2 hit. It must also not suffer a TLB miss, which would initiate a page walk and unnecessarily increase our measured time. This can be done if we choose an array size that is larger than the sum of L1 and L2 cache sizes but smaller than the minimum of (LLC size, L2 TLB reach). This would ensure we are measuring precisely the LLC hit time.

For our machine (refer to 1.1), the L1 and L2 cache sizes are 32KB and 256KB respectively. The LLC size is 16MB and the L2 TLB reach is 6MB. So we chose an array size of 1MB, which would contain 16384 cache blocks. We then accessed these blocks 4096 times each, i.e. a total of 64M cache block accesses and collected both the timings for each block. In the below pseudo-codes, *A* is an allocation of 1MB size:

Define `Block` as a structure with:

integer array of 16 integers `a[16]` ▷ has the same size as a cache block size

Allocate array *A* of type `Block*` containing 16384 Blocks

2.3 Collecting Miss Timings

Algorithm 1 Timing LLC Misses

```

1:  $k \leftarrow 0$ 
2: while  $k \leq \text{NUM-TOTAL-ACCESS}$  do
3:   for  $i \leftarrow 0$  to BLOCKS-IN-PAGE do
4:     for  $j \leftarrow 0$  to MAX-NUM-BLOCKS step BLOCKS-IN-PAGE do
5:        $ptr \leftarrow (\text{char}^*)(A + (i + j) \cdot \text{CACHE-BLOCK-SIZE})$ 
6:       clflush(ptr)
7:       MemoryFence
8:        $start \leftarrow \text{ReadTimestamp}$ 
9:       MemoryFence
10:       $y \leftarrow *ptr$ 
11:      MemoryFence
12:       $end \leftarrow \text{ReadTimestamp}$ 
13:      MemoryFence
14:       $miss\_time \leftarrow end - start$ 
15:      MemoryFence
16:       $k++$ 
17:    end for
18:  end for
19: end while

```

In the above algorithm, k counts the total number of access, and j and i variables denote the Page Number and Block Offset in Page respectively. Each access is done within MemoryFences to ensure that all earlier pending memory actions are done and this action is also completed before we end our measurement. This ensures that we precisely measure the time for this particular access.

2.4 Collecting Hit Timings

For finding the hit times, we would first touch all the blocks to ensure that their Virtual to Physical translations get installed in the TLBs. Since the array size is more than L1+L2 cache sizes, after accessing the entire array, one can expect the blocks from the tail part of the array to be present in L1 and L2 caches more probably than the blocks from the head part. Now when we start our time-access, we start from the beginning, so these blocks will most probably hit in the LLC but miss in L1 and L2. Similarly, when we reach towards the end of the array during access, then L1 and L2 caches will contain more blocks from the start of the array than the end, but LLC would contain them all. So at any point, we have very good chances to get the LLC hit time, even without knowing the replacement policy of the cache.

Algorithm 2 Timing Cache Hits

```

1: for  $i \leftarrow 0$  to BLOCKS-IN-PAGE do
2:   for  $j \leftarrow 0$  to MAX-NUM-BLOCKS step BLOCKS-IN-PAGE do
3:      $A[i + j].a[0] \leftarrow 0$ 
4:   end for
5: end for
6: MemoryFence
7:
8:  $k \leftarrow 0$ 
9: while  $k \leq \text{NUM-TOTAL-ACCESS}$  do
10:  for  $i \leftarrow 0$  to BLOCKS-IN-PAGE do
11:    for  $j \leftarrow 0$  to MAX-NUM-BLOCKS step BLOCKS-IN-PAGE do
12:       $x \leftarrow (\text{char}^*)(A + (i + j) \cdot \text{CACHE-BLOCK-SIZE})$ 
13:      MemoryFence
14:       $start \leftarrow \text{ReadTimestamp}$ 
15:      MemoryFence
16:       $y \leftarrow *x$ 
17:      MemoryFence
18:       $end \leftarrow \text{ReadTimestamp}$ 
19:      MemoryFence
20:       $hit\_time \leftarrow end - start$ 
21:      MemoryFence
22:       $k++$ 
23:    end for
24:  end for
25: end while

```

2.5 Threshold for our machine

In one run of the data collection algorithm, we get 64M data points for hit times and miss times each. For each run, we can find a band of threshold values, which would give minimum errors by performing a sweep. Then the median value of such threshold bands across 5-6 runs can be inferred as a good threshold for LLC hit v/s miss classification. For our machine, we found that

- Typical LLC hit time = 100 rdtsc cycles
- Typical LLC miss time = 300 rdtsc cycles

So, for these configurations, we chose **THRESHOLD** to be **200 rdtsc** cycles, and it gives more than 99.99% accuracy of classification.

3. Receiver: Finding Eviction Set

To isolate a set and then achieving 100% occupancy in that set, the receiver has to identify elements (memory addresses) which map to the same cache set. This would be used by the receiver to evict all the current elements in the cache set and replace them by its own elements. Hence we call it an **eviction-set**. Note that the receiver can isolate any of the cache sets, i.e., the receiver can find elements mapping to any cache set. It would however be better if the cache set that we isolate is less susceptible to noise, or in other words, it is not used by other processes or OS, or by our own processes' data and code blocks.

Challenge : We only have the virtual addresses while the physical addresses are used to index the LLC. The virtual to physical (V2P) address translations are not known hence we cannot get the physical addresses. Even if we get the physical addresses somehow, the hashing function for indexing the cache is also unknown. Therefore, we cannot directly find which cache set any virtual address would map to.

However, note that we do not need to find which cache set a particular element maps to. We only need a way to determine, if given a set of elements, does it evict a cache line from the LLC? If yes, then it contains the eviction set.

3.1 Data Structures & Variables

- *A* and *B*: These are large arrays containing **MAX-NUM-BLOCKS** elements and **MAX-NUM-BLOCKS-B** elements respectively. Each element of this array is of type **Block**. We use this array to find eviction set elements.
- *index_arrayA* and *index_arrayB*: These store the indices of the **Blocks** in *A* and *B* respectively which have shown high miss rate during Eviction set finding.
- *array*: Stores the number of misses that element on the same index in *index_arrayA* has seen till now.
- *receiver_array*: The final eviction set elements (addresses) mapping to the same cache set that receiver has found.
- *istart* and *jstart*: Random offsets within *A* from where we start any exploration.
- *count*: The number of blocks to access in that trial from *istart* and *jstart*.

3.2 Basic Idea

1. If we access a set of cache blocks sequentially, we may observe some misses. These would be due to some cache blocks mapping to the same cache set and these blocks evict each other.
2. If the set of cache blocks is larger, we expect to see more misses. If we access this set of cache blocks repeatedly and if the number of misses is large enough we can conclude that there are a large number of conflicting cache blocks.
3. It is possible that some of these cache blocks map to one cache set while some of them map to some other cache set. However, if we can find at least **ASSOC** cache blocks mapping to the same cache set, we are done, since this would form an eviction set for that cache set.
4. However to test if these map to the same cache set, we would need at least **ASSOC+1** cache blocks, because then it is guaranteed that for each access of this set, at least one would miss in the cache.

3.3 FSM of Eviction Set Finding

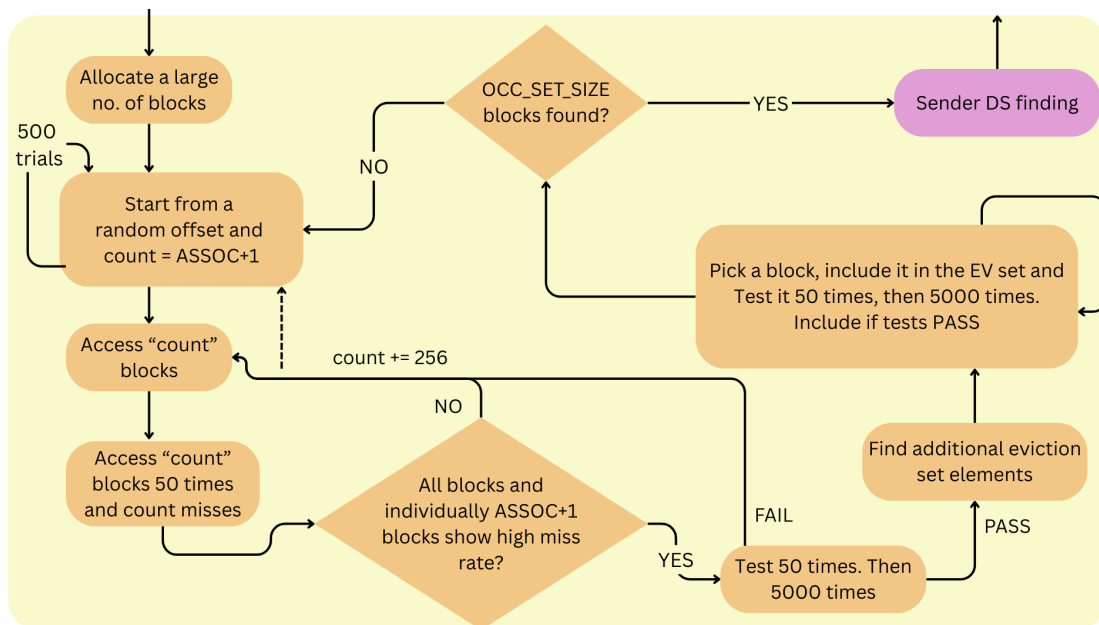


Figure 3.1

3.4 Algorithm to find Eviction Set

Our algorithm (Algorithm 3), in summary works as follows -

1. We use A as a pool of addresses to find eviction set.
2. We take count ($\geq \text{ASSOC}+1$) number of blocks in our set. We access them MIN-NUM-TESTS times as discussed above and count the number of times each block misses.
3. If the total total number of misses reach a threshold of $\text{TESTING-PROB} * \text{MIN-NUM-TESTS}$, we increase the number of tests to MAX-NUM-TESTS .
4. If after the above step, the miss rate is more than $\text{ASSOC}+1$ per test, we suspect that our set may contain $\text{ASSOC}+1$ elements mapping to the same cache set.
5. We sort these blocks in descending order of their number of misses. If the first $\text{ASSOC}+1$ blocks show a miss rate $\geq \text{ISOLATED-TESTING-PROB}$ per test, we send them to an isolated testing function to test if they form an eviction set. If we could not find $\text{ASSOC}+1$ such blocks, we conclude that our set is not large enough to contain an eviction set. So we increase count by $\text{EXPLORATION-BATCH-SIZE}$ and start again. If count reaches MAX-NUM-BLOCKS , we reset count and start a new trial.
6. The test function (Algorithm 5) repeats the same steps as above but now it just accesses these $\text{ASSOC}+1$ blocks. If the test fails we reset count and start a new trial. If the test passes we have found our eviction set. The test function tests for num_tests times and passes if the miss rate per test $\geq \text{SUCCESS-PROB} * \text{number of elements in the set}$.
7. If number of trials reach $\text{MAX-TRIALS-PER-RUN}$, we start a new run and if number of runs reach $\text{EV-SET-FINDING-RUNS}$, we terminate.

Algorithm 3 Eviction Set Construction

```

1: while  $run \leq \text{EV-SET-FINDING-RUNS}$  do
2:   while  $trial \leq \text{MAX-TRIALS-PER-RUN}$  do
3:     Initialize  $count, jstart, istart$ 
4:     while  $count \leq \text{MAX-NUM-BLOCKS}$  do
5:       for  $i \leftarrow 0$  to  $\text{MAX-NUM-BLOCKS}$  do
6:          $A[i].a[0] \leftarrow 0$ ; flush( $\&(A[i].a[0])$ )
7:       end for
8:       Access  $count$  blocks using  $\text{access-pattern}$  from  $istart, jstart$ 
9:        $num\_tests \leftarrow \text{MIN-NUM-TESTS}$ 
10:      for  $t \leftarrow 0$  to  $num\_tests$  do
11:        for  $k \leftarrow 0$  to  $count$  do
12:           $ptr \leftarrow \&A[j * \text{BLOCKS-IN-PAGE} + i].a[0]$ 
13:          Measure access time of  $ptr$ 
14:          if  $time > \text{THRESHOLD}$  then
15:             $miss++$ ;  $(*ptr)++$ 
16:          end if
17:        end for
18:        if enough misses then  $num\_tests \leftarrow \text{MAX-NUM-TESTS}$ 
19:        end if
20:      end for
21:      if  $miss \geq (\text{ASSOC} + 1) \cdot t$  then
22:        Extract non-zero  $A[i].a[0]$  into array and  $index\_arrayA$ 
23:        Sort both arrays in descending order of counts
24:        for  $i \leftarrow 0$  to  $\text{ASSOC}$  do
25:          if  $array[i] < \text{THRESHOLD}$  then break
26:          end if
27:        end for
28:        if  $i == \text{ASSOC} + 1$  then
29:          if test( $\text{MIN-NUM-TESTS}$ ) succeeds then
30:            if test( $\text{MAX-NUM-TESTS}$ ) succeeds then break
31:            else RetryTrial
32:            end if
33:          else RetryTrial
34:          end if
35:        end if
36:      end if
37:       $count \leftarrow count + \text{EXPLORATION-BATCH-SIZE}$ 
38:    end while
39:    goto OccupancySetConstruction
40:  end while
41: end while

```

3.5 Algorithm to Construct Occupancy Set

The receiver needs to construct an occupancy set which it uses in the CRFill+CRProbe step. The size of this set would typically be larger than the eviction set we have found above (i.e., ASSOC+1). Hence, we need to expand our eviction set by finding more elements mapping to the same cache set. We can do this as follows (Algorithm 4) -

1. We use B to find additional elements.
2. Choose any element from B , add it to the set of addresses found till now. If this set still passes the test we include it in our set.
3. We repeat this process until number of elements in our set reaches OCC-SET-SIZE or we have exhausted all elements of B .
4. If we have exhausted all the elements of B and still could not find OCC-SET-SIZE many blocks, we go for the next trial of finding eviction set.

Algorithm 4 Occupancy Set Construction

```

1:  $i \leftarrow 0, j \leftarrow 0$ 
2:  $index\_arrayB[j] \leftarrow i$ 
3: while  $i < \text{MAX-NUM-BLOCKS-B}$  do
4:   if  $\text{test}(j + 1, \text{MIN-NUM-TESTS})$  and  $\text{test}(j + 1, \text{MAX-NUM-TESTS})$  then
5:      $j \leftarrow j + 1$ 
6:     break
7:   end if
8:    $i \leftarrow i + 1$ 
9:    $index\_arrayB[j] \leftarrow i$ 
10: end while
11: if  $i < \text{MAX-NUM-BLOCKS-B}$  then
12:    $count \leftarrow 1, j \leftarrow j + 1, i \leftarrow i + \text{BLOCKS-IN-PAGE}$ 
13:   while  $j \neq \text{OCC-SET-SIZE}$  and  $count < \text{MAX-NUM-BLOCKS-B}$  do
14:      $index\_arrayB[j] \leftarrow i$ 
15:     if  $\text{test}(\text{MIN-NUM-TESTS})$  and  $\text{test}(\text{MAX-NUM-TESTS})$  then
16:        $j \leftarrow j + 1$ 
17:     end if
18:      $count \leftarrow count + 1, i \leftarrow i + \text{BLOCKS-IN-PAGE}$ 
19:   end while
20:   if  $j = \text{OCC-SET-SIZE}$  then
21:     break
22:   else RetryTrial
23:   end if
24: else RetryTrial
25: end if

```

3.6 Algorithm to Test if a set is an eviction set

We have a separate Test function which returns 1 if the input set of addresses is an eviction set for some set in the cache, 0 otherwise. It performs the same test as the receiver itself performed before calling the test function, but its passing threshold is higher here. Also, it gives a different stack state and a different code block altogether, ruling out the chances of any code block of main showing false misses there.

Algorithm 5 TestEvictionSet(num_tests)

```

1: for  $i \leftarrow 0$  to MAX-NUM-BLOCKS do
2:    $A[i].a[0] \leftarrow 0$ ; flush(&(A[i].a[0]))
3: end for
4: for  $t \leftarrow 0$  to num_tests do
5:   for  $i \leftarrow 0$  to ASSOC do
6:      $ptr \leftarrow \&A[index\_arrayA[i]].a[0]$ 
7:     Measure access time of  $ptr$ 
8:     if time > THRESHOLD then
9:       miss++;
10:      if num_tests = MAX-NUM-TESTS then (*ptr)++
11:      end if
12:    end if
13:  end for
14: end for
15: if num_tests = MAX-NUM-TESTS and miss is sufficient then
16:   Extract nonzero  $A[i].a[0]$  to array, sort by count
17:   if top ASSOC+1 counts are large enough then
18:     Reflush and retest
19:     if miss is sufficient then return 1
20:   end if
21: end if
22: end if
23: return 0

```

3.7 Validating Eviction Set

Once we have found our eviction set¹, we want to check if it is susceptible to noise. For this we perform the validation step -

1. The receiver performs the CRFill + CRProbe step.
2. It then performs the communication step, as though it is receiving CHECK-STRLEN bits (all 0s). However there is no sender involved here.
3. Since there is no disturbance from the sender, the receiver should find all hits. Any miss observed would be because of noise.
4. If it observes a miss, that bit is inferred as a 1, hence it is an error. It counts the number of errors and calculates the BER.
5. If $BER \geq \text{SELF-VALIDATION-ERROR-THRESHOLD}$, we conclude that the cache set isolated is not quiet enough. Hence, the receiver notifies the sender and goes on to find a new eviction set.

¹From now on, whenever we use the term "eviction set", it refers to this expanded eviction set which we referred to as the "occupancy set" above. This is the set used by the receiver to perform CRFill+CRProbe step.

4. Sender: Finding Disturbance Set

Once the receiver has finalized the [eviction-set](#), the sender needs to find elements mapping to the same cache set for which receiver has found the eviction set. These elements would form the [disturbance-set](#).

Challenge : Sender is a different process. Therefore, it does not have access to the receiver's eviction set elements. Therefore, sender cannot find disturbance set on its own. Involvement of receiver is required.

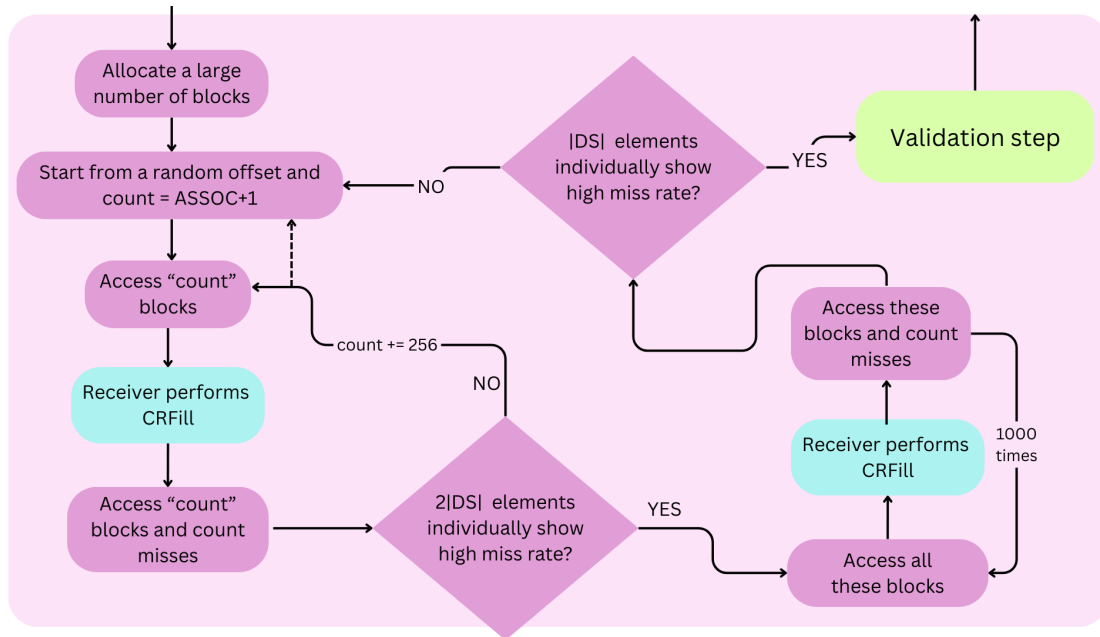
4.1 Algorithm to find Disturbance Set

The idea of the algorithm is similar to what we did for expanding the eviction set. The algorithm works as follows -

1. Sender allocates an array *A* (similar to receiver) which acts as a pool of addresses to find disturbance set elements.
2. It accesses count number of blocks, i.e. in a way "primes" the LLC with its blocks. This can be seen as though sender is *randomly bombarding* to create some disturbance in the receivers' blocks.
3. Then the receiver performs CRFill step to evict any blocks the sender might have filled in the cache set it had isolated.
4. The sender then time accesses all of its blocks to find which blocks missed. It is likely that these blocks were evicted by the receiver and hence are potential elements of the disturbance set.
5. However, note that, if count is large, some of the misses could be due to self evictions. Also some misses could be there because of noise. Therefore, we move to the next step if we get atleast $2 \times DS$ elements which miss in the cache, hoping that atleast *DS* elements were actually evicted by the receiver. If we did not observe $2 \times DS$ misses, we believe that count is not large enough and therefore increase it by [EXPLORATION-BATCH-SIZE-SNDR](#), then go back to step 2.
6. If we get $2 \times DS$ potential elements, we test these [MAX-NUM-TESTS](#) times.
7. In each iteration, steps 2 to 4 are repeated, and sender records the number of misses for each potential element. Note that this time, since we have a smaller number of elements, self-evictions are unlikely. Although there can be some evictions due to noise.
8. After above step, if we find that *DS* elements have number of misses greater than [ISOLATED-TESTING-PROB](#) * [MAX-NUM-TESTS](#), then we have found our disturbance set.

9. If we could not find **DS** with high enough miss rate, then we start a new trial, i.e., reinitialise count and start again from step 2.
10. The number of trials is upper bounded by **DS-FINDING-TRIALS**. If we could not find a potential disturbance set in these many trials, we terminate.

4.2 FSM for finding the Disturbance set



5. Validating Disturbance Set & Eviction Set

We have found our eviction set and disturbance set. However, there is a possibility that the disturbance set is not correct, or that the cache set we have isolated is more susceptible to noise when sender and receiver communicate. To avoid such a case we validate our disturbance set and eviction set -

- We have a train suite which contains **TRAIN-SUITE-LEN** strings of **STRLEN** bits each.
- Sender communicates all the strings in this train suite to the receiver in the same way they would communicate¹ finally (refer to 7). Only difference is that the train suite is known to receiver as well.
- Receiver keeps counting the number of bits it infers incorrectly.
- Receiver checks if the overall **BER** is less than **VALIDATION-ERROR-THRESHOLD**, we accept the **eviction-set** and the **disturbance-set** and move to actual communication.
- Otherwise, we discard both the sets and go back to finding eviction set.

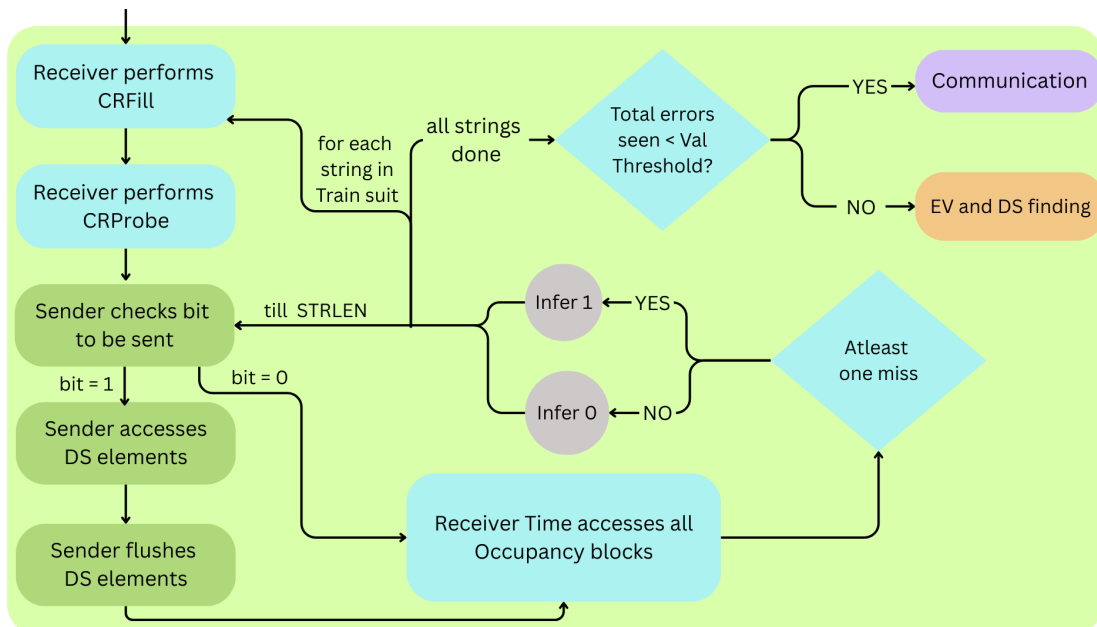


Figure 5.1: FSM for validation of the **DS** and **EV** set

¹We assumed an unrolling factor of 1 and **DS** = 4 for this validation (refer to 6)

6. Unrolling Factor Threshold

6.1 What is Unrolling Factor?

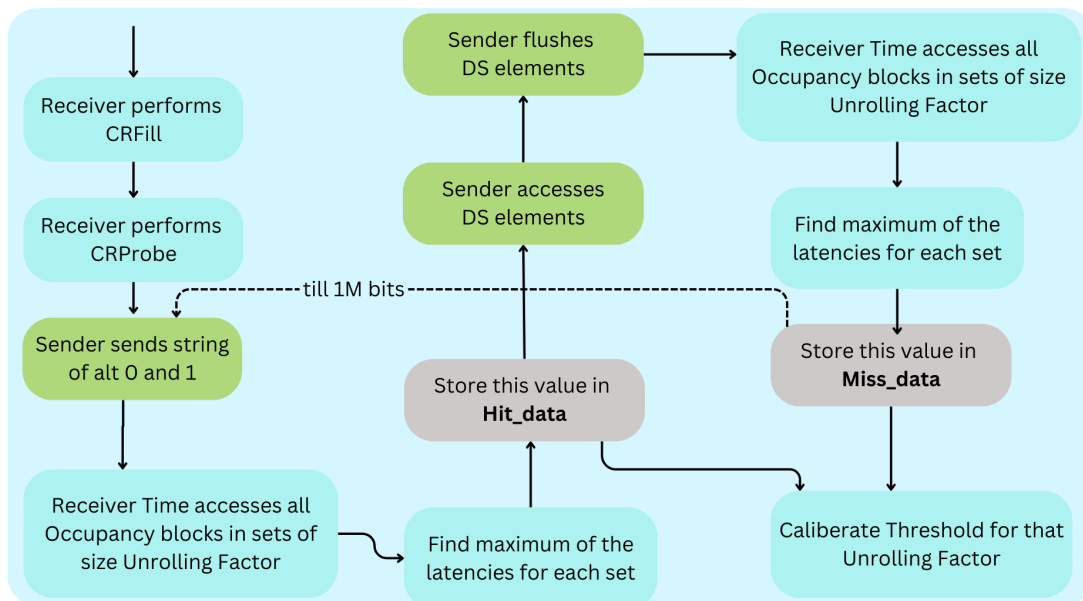
In order to correctly infer a bit sent by the sender, the receiver needs to be able to distinguish between all hits v/s atleast one miss when it time-accesses its blocks. We need to do each access within MemoryFences, to ensure that all previous memory operations were completed before we started measuring, and this operation was completed before we stop measuring. But this would lead to a total of $|DS|$ sequential miss timings in the worst case. Although, we could have correctly inferred a bit 1 even on the first miss. An alternate way to speed up this would be to access blocks in bunches, and time the total time of access of that bunch. Doing this would allow us to have atleast some misses being served in parallel and therefore the overall time would be reduced. But now we need another threshold, which should be able to distinguish between all hits in the bunch, v/s atleast one miss. This is what we would call the Threshold for that Unrolling Factor, where **Unrolling Factor** is the size of the bunch we are measuring in one go. Doing this would therefore help us improve the bandwidth significantly without compromising much on the bit-error-rate.

6.2 Finding Threshold for any Unrolling Factor

The basic idea of the algorithm is as follows:

1. The receiver wants to find a threshold which would give minimum errors by sweeping over 1M data points, stored in the array `thresh_data`.
2. To collect this data, the sender sends a string of alternate 0s and 1s, and the receiver records the timings for both and fills those in its array `thresh_data`.
3. If the iteration count is even, sender would have sent a 0, hence receiver must treat the data point as hit time. Similarly, for odd iteration count, the receiver must treat it as miss time.
4. For each data point, the receiver must only store the max observed time across all bunches. For example, if **Unrolling Factor** is 2, then we would have $ASSOC/UF = 16/2=8$ bunches, having 2 cache blocks each. Now, the max observed timing across all these 8 independent measurements would be essential to decide all hit v/s atleast one miss. Hence, the receiver would just store the max in the array `thresh_data`.

6.3 FSM for finding this threshold



7. Communication

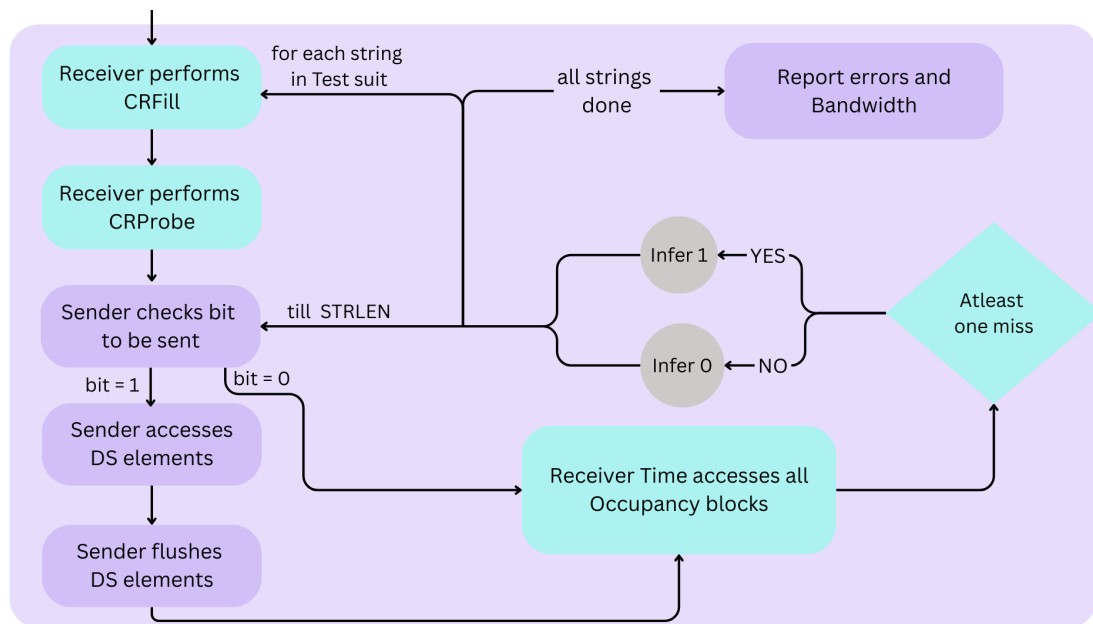
7.1 Basic Idea

Now that our covert channel is established, we are all set to perform the final step, communication of strings. We have used a Test suite, which is set of **TEST-SUITE-LEN** random strings, each of **STRLEN** bits, for communication. The steps of communication for any string are outlined as follows:

1. The receiver performs the **CRFill** step. This step allows it to isolate and prepare that cache set by filling blocks from its eviction set into it and evicting those already present there.
2. The **occupancy-sequence** we have used is $(1x)(1.1x)^2(1.2x)^6$, which gives more than 99% chances of occupying any cache set.
3. Then the receiver performs the **CRProbe** step. This step allows it to find out actually how many blocks are present in the cache set, after performing the **CRFill** step. For this, the receiver starts from the last block it had accessed (this must be a hit, unless the block is evicted due to noise) and goes in the reverse order of accessing blocks than that in **CRFill**.
4. Each block which shows a hit during **CRProbe** step is added to the set of occupied blocks. So the final number of occupied blocks is the number of blocks that hit in the cache after **CRFill**. (those which show a miss would occupy some hole we have created earlier as we flush each block after accessing it in this step)
5. Now the cache set is prepared for communication of the string, and sender will be signaled to start sending. Till now the sender was waiting, but now its time for the receiver to wait.
6. The sender, if has to send a bit 0, would create no disturbance in the cache set, and tell the receiver to start inferring. Otherwise, the sender would access elements from its disturbance set, and evict atleast one receiver's block (assuming 100% occupancy). Now the sender must wait and receiver will work.
7. The receiver will time access blocks in bunches (of size **Unrolling Factor**) and if any of these timings was greater than the Threshold for that **Unrolling Factor** which they had earlier calibrated, then it will infer a bit 1. Otherwise if all are less, then it will infer a bit 0.
8. Now the receiver goes to wait and tells sender to start sending the next bit, till all bits in the string are sent.

This process is repeated for each string to be sent.

7.2 FSM for communication



After the communication of all the strings is done, we report the total number of errors seen, and the average Bandwidth of communication.

8. Assumptions & Scope of Improvement

8.1 Synchronisation

Currently, as we discussed before briefly, we have synchronised the sender and receiver using pipes. This synchronisation is necessary for our channel to work since it must be ensured that the sender and receiver are not working simultaneously at any point. However, synchronisation with pipes assumes some sharing of memory between the sender and receiver processes which is against idea of establishing a covert channel since shared memory is a legitimate way of communication between processes. Although, we do not use pipes to share any bits during actual communication, some communication via pipes is needed in earlier steps. For eg., the receiver has to notify the sender if the validation passed or failed. However, in most of the other cases, this assumption of shared pipes can be lifted by considering the worst case timing for each step. For example, if the receiver takes (say) 1000 cycles for a step in the worst case, then we can make the sender wait for 1000 cycles irrespective of how many cycles receiver took this particular time, i.e., if the receiver took only 800 cycles this time, the sender would still wait for 1000 cycles and the receiver would also wait for 200 cycles.

It is apparent that this is a more difficult task, since worst case timing for each step must be measured precisely. If in any case the actual timing goes more than the estimated worst case time, the sender and receiver will become asynchronous and the channel will not work. One can think of using a large worst case time to be safe, but this would make the channel slower and would also increase chances of interference due to OS noise. Synchronisation without pipes is a major improvement which can be done in the future.

8.2 Privileges

- We had root privileges on our machine which we used to pin the sender and receiver processes to different cores and set high affinity. This reduces the possibility of context switches.
- We ensured that no other user process was running (except for necessary background services). This ensures low noise from other processes.
- The machine was booted in single user mode to avoid disturbance from other users.

9. Evaluation Metrics

9.1 Data Collection

We have collected data for various values of unrolling factor (UF) and disturbance set size (DS).

- UF = 1, 2, 4, 8, 16
- DS = 1, 2, 4

This gives a total of 15 configurations, i.e., 15 different (UF,DS) tuples. For collecting the data we sped up the process by reusing the eviction set and disturbance set. Once we have found an eviction set and disturbance set, we use these to communicate the test suite and collect the bandwidth and BER data for each configuration in the same run. This eliminates the need of repeating the time consuming steps of finding eviction set and disturbance set. We collected data for 8 such runs and the data presented here is average across all the 8 runs. We measured all the timings in cycles using rdtsc instruction in x86 ISA.

9.2 Bandwidth

9.2.1 Calculation

We excluded the time taken by the pipe read and write syscalls while calculating the bandwidth. Calculation of bandwidth was done as follows -

- Let the cycles for CRFill + CRProbe of k^{th} string be T_{CR}^k cycles.
- Let the cycles taken by sender to send the i^{th} bit of the k^{th} string be S_i^k cycles.
- Let the cycles taken by receiver to receive (infer) the i^{th} bit of the k^{th} string be R_i^k cycles.
- For N bit string, the worst case cycles to send a bit of k^{th} string, $T_S^k = \max_{1 \leq i \leq N} \{S_i^k\}$ cycles.
- For N bit string, the worst case cycles to receive (infer) a bit of k^{th} string, $T_R^k = \max_{1 \leq i \leq N} \{R_i^k\}$ cycles.
- Total cycles for communication of N bits of k^{th} string, $T_{COMM}^k = N \times (T_R^k + T_S^k)$ cycles.
- Total cycles taken for k^{th} string, $T_C^k = T_{CR}^k + T_{COMM}^k$ cycles.
- Let the number of strings in test suite be M .
- Total cycles taken for the test suite, $T_C = \sum_{k=1}^M T_C^k$

- Let the Clock Frequency be F cycles per second.
- Total time taken, $T = \frac{T_c}{F}$ seconds
- Total number of bits communicated, $b = N \times M$.
- Bandwidth, $B = \frac{b}{T} = \frac{N \times M}{T} = \frac{N \times M \times F}{T_c}$ bps

In our case, we had Clock Frequency, $F = 3.4 \text{ GHz} = 3.4 \times 10^9$ cycles per second, $N = 512$, $M = 500$. Thus, bandwidth B can be given by

$$B = 512 \times 500 \times 3.4 \times 10^9 \times \frac{1}{T_c} \text{ bps} = \frac{8.704 \times 10^{11}}{T_c} \text{ Kbps}$$

9.2.2 Results & Inferences

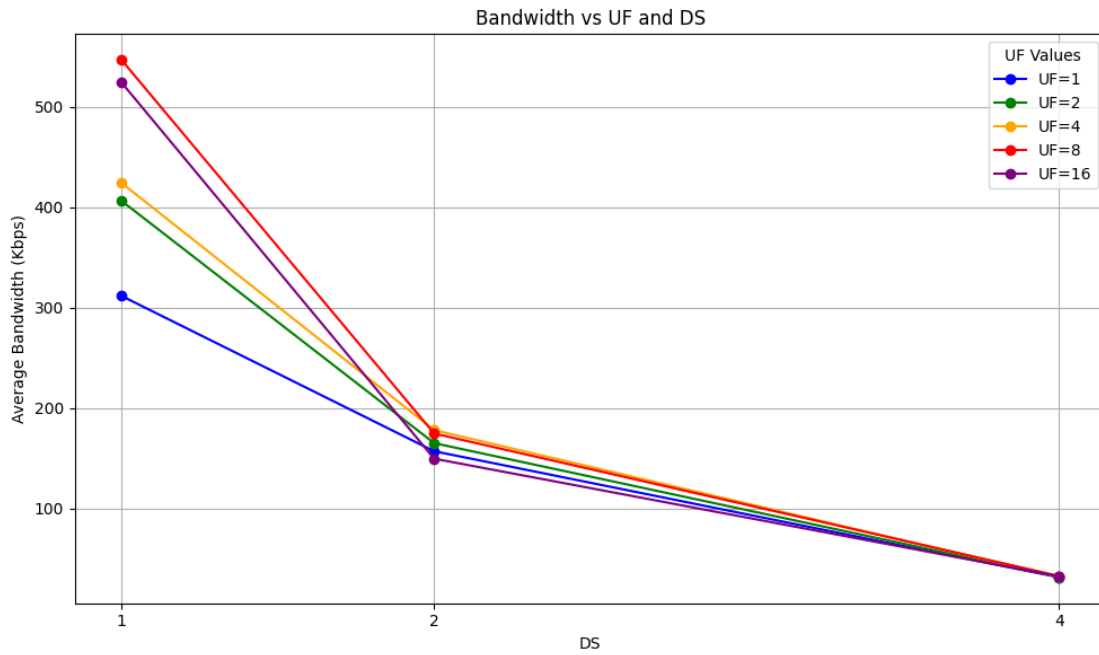


Figure 9.1: Average Bandwidth vs DS for UF=1,2,4,8,16

We can observe that the **bandwidth decreases significantly with increasing DS**. Also, we can see a **significant increase in bandwidth on increasing the unrolling factor**.

9.3 BER

9.3.1 Calculation

Calculation of BER is straight forward -

- Let total number of errors in communication of complete test suite be E .
- Let the number of strings in test suit be M and length of each string be N bits.
- Total number of bits communicated, $b = N \times M$.
- $BER = E/b$

In our case, $N = 512$ and $M = 500$. Therefore, $b = 512 \times 500 = 25600$. Therefore BER is given by

$$BER = \frac{E}{25600} = E \times 3.90625 \times 10^{-5}$$

9.3.2 Results & Inferences

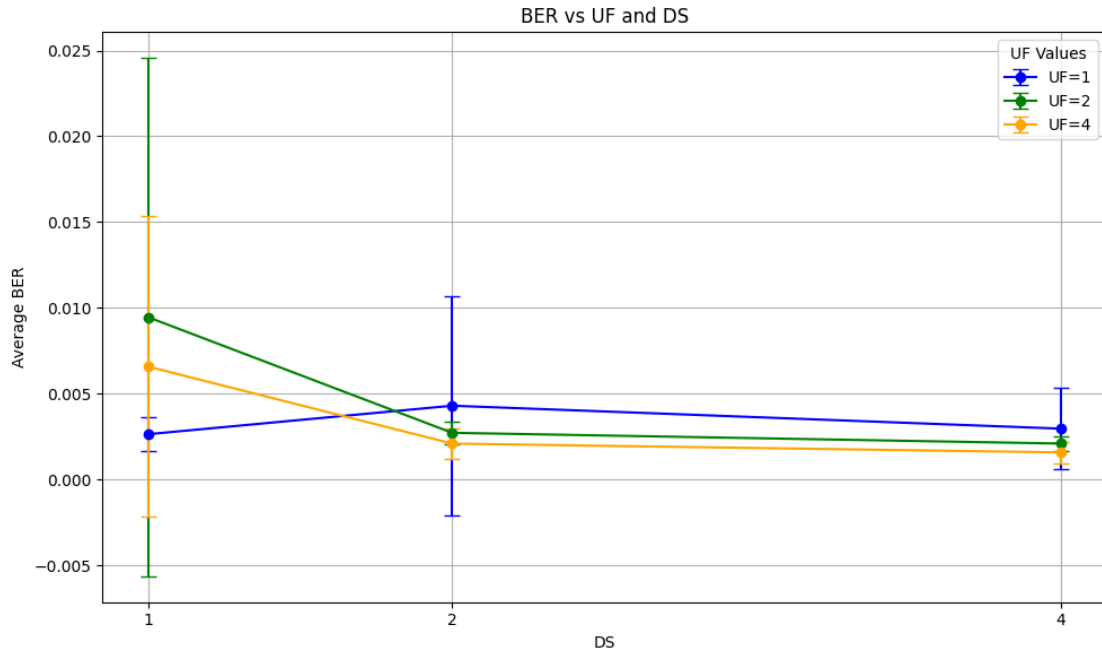


Figure 9.2: Average BER (and std.dev.) vs DS for UF=1,2,4

We can observe that it is hard to find a perfect correlation between unrolling factor and BER. Similar is the case for DS and BER. However, there is a roughly a general trend - BER decreases with increasing DS. Also there is a large variation in BER. This is because the BER depends heavily on the eviction set and disturbance set we have found and hence varies across runs due to noise.

9.4 Is large UF better?

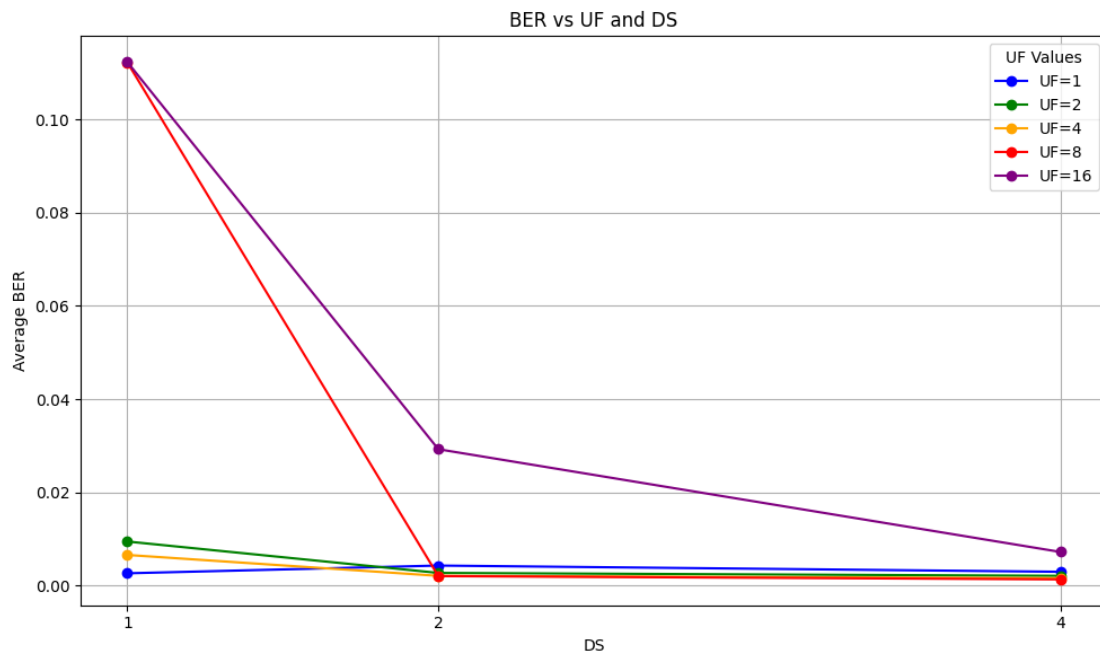


Figure 9.3: Average BER vs DS for UF=1,2,4,8,16

For larger UF values, like 8 and 16, we can see a significant increase in bandwidth. However, if we consider BER, they show a large BER in some of the cases which skew the average. This may be due to the fact that for a larger UF, the overlap in latencies of all hits vs at least one miss may become so large, that getting a clear threshold is not possible, leading to a large number of errors. Most of the errors for UF = 1,2,4 are 0 to 1 errors, which can be explained by noise. This is also true to some extent for UF = 8, 16 where number of 0 to 1 errors is as expected. But in some runs, we find a large number of 1 to 0 errors which could only be explained by a poor threshold and a large overlap in latencies. Thus, while UF = 8, 16 have a higher bandwidth, they show a low fidelity and hence are not suitable for communication.

Glossary

access-pattern The pattern of accessing an allocation of pages, jumping across pages for each block, as shown in figure [2.1](#).

ASSOC Number of ways in the LLC.

BER Number of error bits as a fraction of the total number of bits sent.

Block a struct of 16 integers so that its size is same as a cache block size, i.e., 64B.

BLOCKS-IN-PAGE Number of cache blocks that fit in a page, which is $(\text{Page size})/(\text{Cache Block size})$, 64 in our case.

CACHE-BLOCK-SIZE Size of one cache block in bytes. Typically it is 64 bytes.

CHECK-STRLEN 1,000,000.

CRFill The step of filling one's own blocks into a cache set and evicting already present ones..

CRProbe After the CRFill step, finding how many blocks are actually present in the cache set.

disturbance-set Set of addresses used by the sender to create disturbance in the LLC cache set isolated by the receiver.

DS The size of disturbance set.

DS-FINDING-TRIALS 1000.

EV-SET-FINDING-RUNS 20.

eviction-set Set of addresses used by the receiver to evict a cache line from the LLC..

EXPLORATION-BATCH-SIZE 256.

EXPLORATION-BATCH-SIZE-SNDR 256.

ISOLATED-TESTING-PROB 0.9.

MAX-NUM-BLOCKS The number of cache blocks allocated by the receiver in memory for finding eviction-set.

MAX-NUM-BLOCKS-B The number of cache blocks allocated by the receiver in memory for finding additional elements to expand the eviction set to OCC-SET-SIZE.

MAX-NUM-TESTS 1000.

MAX-TRIALS-PER-RUN 500.

MIN-NUM-TESTS 50.

NUM-TOTAL-ACCESS 2^{26} .

OCC-SET-SIZE Number of elements required by the complete occupancy-sequence, $\text{ceil}(1.2 \cdot \text{ASSOC}) = 20$ in our case.

occupancy-sequence Set of addresses which the receiver accesses to fill them in its cache set. It can be expressed as $(A_1x)^{B_1}(A_2x)^{B_2}\dots$, which means that $A_i \cdot \text{ASSOC}$ addresses are accessed B_i times, starting from the beginning each time.

SELF-VALIDATION-ERROR-THRESHOLD $4 \cdot 10^{-3}$.

STRLEN 512.

SUCCESS-PROB 0.99.

TEST-SUITE-LEN 500.

TESTING-PROB 0.8.

THRESHOLD The value of rdtsc cycles used to differentiate between LLC hit and LLC miss. If the measured time is less than or equal to the threshold, it is considered an LLC hit. If the measured time is greater than the threshold, it is considered an LLC miss..

TRAIN-SUITE-LEN 250.

UF Unrolling Factor.

Unrolling Factor The number of cache blocks in units of which the receiver will time-access the cache set, as a bunch, during communication.

VALIDATION-ERROR-THRESHOLD $6 \cdot 10^{-4}$.

Bibliography

- [1] Fangfei Liu et al. “Last-Level Cache Side-Channel Attacks are Practical”. In: *2015 IEEE Symposium on Security and Privacy*. 2015, pp. 605–622. doi: [10.1109/SP.2015.43](https://doi.org/10.1109/SP.2015.43).
- [2] Pepe Vila, Boris Köpf, and José F. Morales. “Theory and Practice of Finding Eviction Sets”. In: *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 39–54. doi: [10.1109/SP.2019.00042](https://doi.org/10.1109/SP.2019.00042).